

INTERACTIVE WEB PROJECT

無限井字遊戲： 動態配對與即時同步架構

國立澎湖科技大學

資工二甲

林柏翰

簡報大綱 目錄

01 專題研究背景與動機

02 無限規則與遊戲機制

03 技術架構：Firebase 應用

04 核心技術：連線配對邏輯

05 挑戰攻克：斷線與幽靈房

06 視覺設計與 UI/UX 優化

07 結論與未來展望

研究背景與動機

傳統井字棋侷限

棋盤空間有限 (3x3)，熟練玩家容易進入「必和局」的死胡同，缺乏變數與趣味性。

無限規則構想

引入印記上限與自動消除機制，棋盤狀態動態輪轉，挑戰玩家的長期佈局策略。

遊戲核心機制：三印記限制

印記存留與 FIFO

每位玩家場上最多保留 3 個符號。放置第 4 個印記時，系統執行 **shift()** 取出最舊座標並自動消失。

連線獲勝判定

系統在放置印記瞬間優先執行空間掃描判定（橫、直、斜 8 種可能）。若判定獲勝則寫入 **status: 'finished'**，否則才執行 FIFO 消除。

動態性與視覺反饋

戰局永遠處於變化中，無法產生絕對平手。這項機制徹底解決了傳統井字棋容易產生平手的缺點。為了優化使用者體驗，我們設計了視覺預警系統

技術架構：Cloud-Based Sync

Firebase RTDB

核心數據中樞。透過 WebSocket 達成「事件驅動」的即時同步，取代傳統輪詢。

Module JS

前端邏輯封裝。將大廳配對與遊戲本體解耦，利用 sessionStorage 進行狀態傳遞。

Web Routing

基於 URL 解析的動態導向，支援不同資料夾結構下的路徑自動校正。

Timestamping

伺服器端時間紀錄，確保倒數計時與離線偵測的精準度。

配對邏輯：原子化交易

為解決多用戶同時存取產生的 Race Condition，採用 **runTransaction** 機制：

```
runTransaction(gameRef, (data) => {  
  if (!data || data.status === 'finished') {  
    return { status: 'waiting', hostId: myID };  
  } else if (data.status === 'waiting') {  
    data.status = 'playing';  
    return data;  
  }  
});
```

- > 房主角色：第 1 人初始化數據，狀態變為 waiting。
- > 挑戰者：第 2 人將狀態變更為 playing。
- > 自動跳轉：雙方監聽到狀態改變後，同步載入棋盤頁面。

挑戰：幽靈房間與重置循環

幽靈房間問題

當玩家直接關閉分頁，Firebase 數據依然殘留，導致下一位玩家進入「死房」。

解法：導入 `onDisconnect().remove()`，達成「人走房毀」。

無限重連迴圈

跳轉與監聽器重疊觸發衝突。

解法：使用 `unsubscribe()` 立即卸載大廳監聽器，並配合 300ms 隨機延遲。

介面設計與 UX 反饋

- > 響應式棋盤：Grid 佈局自適應不同設備螢幕。
- > 角色辨識：明確標示「你是藍方」或「你是紅方」，解決操作困惑。
- > 狀態同步：整合遊戲紀錄、悔棋次數、與回合計時器。
- > 防呆機制：非當前回合玩家點擊無效，並給予視覺提示。

等待對手加入中...

已等待：6 秒

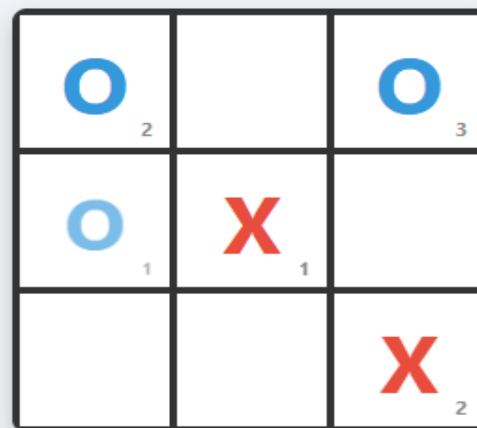
無限井字遊戲

目前回合：紅方 X 【你是：藍方 O】

回合數：5

時間：00:28

悔棋次數：藍方 0 / 紅方 0



悔棋

重新開始

專題總結與心得

- > 技術整合：成功將 Firebase NoSQL 資料庫與前端 JavaScript 深度結合。
- > 邏輯驗證：透過「無限規則」設計，驗證了複數狀態機在即時系統中的運作效率。
- > 穩定運作：克服了 WebSocket 環境下的連線生命週期管理，達成自動化配對循環。

未來展望



多房模式

建立房間列表系統，支援多組玩家同時在不同房間進行對戰。



AI 對戰模式

引入 Minimax 演算法，在離線或無對手時提供單機練習模式。

THANK YOU

感謝您的聆聽與指教

(Q&A)
